Quantum Machine Learning

Paul Caplin, Belle Cowan, and Tuhin Kr Mondal

Group name: Quantum Chargers 2
University of Alabama in Huntsville
Mentor: Dr. Dinh Nguyen

Abstract

Quantum machine learning represent an emerging approach to machine learning that integrates principles of quantum computing with traditional neural network architectures. This project explores the application of quantum Neural Networks (QNNs), a QML model for image classification using two widely recognized benchmark datasets: MNIST and CIFAR-10. The MNIST dataset consists of 70,000 grayscale images of handwritten digits, each with a resolution of 28 × 28 pixels, while CIFAR-10 contains 60,000 color images of size 32 × 32 pixels distributed across ten object categories. The proposed approach utilizes quantum computing concepts, including superposition, entanglement, and parameterized quantum circuits, to investigate their potential for learning and representing complex image features. Image data are processed and used to train a QNN to distinguish between different image classes. During training, the model optimizes its learnable parameters to minimize classification loss and improve prediction accuracy. The performance of the QNN is evaluated by monitoring classification accuracy and loss across training epochs, with the results visualized through performance graphs. Furthermore, to investigate the robustness of the proposed model under more realistic quantum computing conditions, measurement noise is incorporated into the simulation framework. A finite-shot measurement strategy is then implemented to examine its impact on training stability, convergence behavior, and classification performance. Simulation results demonstrate that increasing the number of measurement shots improves the training accuracy of the QNN model by reducing statistical fluctuations in quantum measurements, leading to more stable and reliable learning. Overall, this study demonstrates the feasibility of QNNs for image classification and highlights the importance of measurement shot configuration in enhancing training performance and robustness under practical quantum computing constraints.

I. Introduction

Quantum Machine Learning (QML) represents an emerging area of research that combines quantum computing techniques with traditional machine learning methods to explore new possibilities for solving complex learning problems. Through quantum phenomena such as superposition and entanglement, QML provides alternative approaches to information representation, data processing, and computational optimization. Among the various QML models, Quantum Neural Networks (QNNs) have received increasing attention due to their ability to incorporate trainable quantum circuits into machine learning frameworks. QNNs typically consist of quantum feature encoding layers, parameterized quantum gates, and measurement operations that transform input data into outputs suitable for prediction and classification. The trainable parameters of these circuits are adjusted using classical optimization algorithms to minimize a loss function and improve model performance. In image classification applications, classical image data can be preprocessed and encoded into quantum states, allowing QNNs to learn representations through quantum circuit operations. However, implementing QNNs for practical classification tasks introduces several challenges, including limited quantum hardware resources, circuit complexity, and noise during quantum operations and measurements. Measurement uncertainty is particularly important because quantum circuit outputs are estimated from a finite number of measurements, which can introduce fluctuations in predictions and affect the optimization process. Increasing the number of measurement shots provides a practical approach to reducing statistical uncertainty and obtaining more reliable estimates, although it requires additional computational resources and execution time. Motivated by these challenges, this project focuses on the implementation and evaluation of QNNs for image classification using real-world datasets. The developed framework combines classical image preprocessing, quantum data encoding, variational quantum circuits, and classical parameter optimization to investigate the performance of quantum-based classification models. In addition, the study explores how different measurement shot counts influence classification accuracy, training loss, and convergence behavior. The main objective is to examine the feasibility, effectiveness, and practical limitations of QNNs in real-world image classification tasks.

A. Quantum Neural Network Overview

Quantum neural networks (QNNs) are machine learning models that use quantum circuits to transform information and learn patterns in data. These circuits operate on qubits through quantum gates. Unlike a classical bit, which represents either zero or one, a qubit can exist in a superposition of both states. Multiple qubits can also become entangled, creating correlations between their measurement outcomes. QNNs use these properties to construct trainable transformations of input features. Researchers investigate these models for classification, optimization, and other computational tasks, although faster computation or higher accuracy than classical models is not guaranteed.

The model in our program uses a hybrid quantum-classical architecture to distinguish handwritten digits 3 and 6 from the MNIST dataset. Classical operations prepare the images, calculate the training loss, and update the model parameters. A

four-qubit quantum circuit processes the image features. The model's forward pass has three main stages: encoding, quantum processing, and measurement. During training, these stages are repeated as the optimizer adjusts the circuit's rotation angles to improve classification. This implementation simulates the quantum circuit on a classical CPU or GPU.

B. Encoding

Encoding converts classical image features into a quantum state. The code first converts each MNIST image into a numerical tensor and normalizes its pixel values using the specified dataset mean and standard deviation. It then applies average pooling with a window size and stride of six. This operation averages groups of pixels, reducing each 28-by-28 image to a 4-by-4 feature grid. The grid is flattened into a sequence of 16 values that supplies the inputs to the quantum encoder.

The model uses angle encoding, meaning that the feature values determine the rotation angles of quantum gates. Each of the four qubits receives four consecutive values from the flattened sequence. Three values control a U3 gate, which performs a general single-qubit rotation, and the fourth controls an RX gate, which rotates the qubit around the Bloch sphere's x-axis. Hadamard gates are also included to mix the amplitudes associated with zero and one. Their position in the encoding sequence differs: the Hadamard follows the U3 and RX gates on qubit 0 and precedes them on the other three qubits.

$$\lfloor \psi \rangle = \bigotimes_{i=0}^{N-1} R_Y(x_i)|0\rangle]$$

All four qubits initially start in state zero. Applying the encoding gates prepares an image-dependent quantum state that becomes the input to the trainable circuit. The encoding angles come directly from the image features, so different images produce different initial states for quantum processing.

C. Quantum Circuit

After encoding, the circuit transforms the quantum state using a sequence of rotations and controlled operations. When the program runs through its main function, it constructs an initial layer containing 50 randomly selected RX, RY, RZ, and CNOT gates. RX and RY gates change the state through rotations around their respective axes, while RZ gates adjust its relative phase. CNOT gates connect pairs of qubits and can create entanglement, allowing features encoded on different qubits to interact.

The random gate sequence is generated when the model is constructed. Its gate types, order, and qubit connections remain fixed during training, while its parameterized rotation angles can change. The model therefore learns by adjusting how strongly particular rotations act, rather than repeatedly generating a new circuit structure.

Following the initial layer, the circuit applies additional trainable rotations: RX on qubit 0, RY on qubit 1, and RZ on qubit 3. It then applies a controlled RX operation with qubit 0 controlling qubit 2. This operation rotates the target qubit conditionally on the control being in state one. Finally, the circuit applies a Hadamard gate to qubit 3, an SX gate to qubit 2, and a CNOT from qubit 3 to qubit 0. The SX gate is a square root of the X gate. Together, these operations form the model's quantum feature transformation, whose outputs are passed to the measurement stage.

D. Measurement of the QNN Model

The measurement stage extracts classical information from the processed quantum state. A four-qubit state contains 16 complex amplitudes, one for each possible combination of four binary outcomes. The simulator squares the magnitudes of these amplitudes to obtain their probabilities. For each qubit, it combines the relevant probabilities to determine the probability of measuring zero and the probability of measuring one.

The model then calculates a Pauli-Z expectation value for each qubit by subtracting its probability of one from its probability of zero. This produces four continuous values between negative one and positive one. A value near positive one indicates that zero is more likely, while a value near negative one indicates that one is more likely. A value near zero indicates approximately equal probabilities.

The model keeps the expectation values from qubits 0 and 1 as its two classification scores. It applies log-softmax to convert these scores into normalized log probabilities. The first output represents digit 3, and the second represents digit 6. The larger output determines the predicted class.

During training, negative log-likelihood loss compares the outputs with the correct labels. Because the simulator calculates expectation values directly from the quantum state, PyTorch can differentiate through these calculations and obtain gradients for the circuit parameters. Adam uses those gradients to update the trainable angles. The script performs this process over 100 epochs using batches of 64 images and a learning rate of 0.001.

II. Simulation Results

A. QNN Model Training without Measurement Noise

The figures below show the results we get from 100 epochs on each dataset. We trained our model on two different datasets: the MNIST database (Modified National Institute of Standards and Technology) and the CIFAR-10 dataset (Canadian Institute for Advanced Research). MNIST is a set of 70,000 images containing handwritten digits from 0 to 9. These images are 28x28 pixels and in grayscale. CIFAR-10 is a set of 60,000 low resolution images containing 10 classes of images: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each class contains 6,000 images and each image is 32x32 pixels and colored.

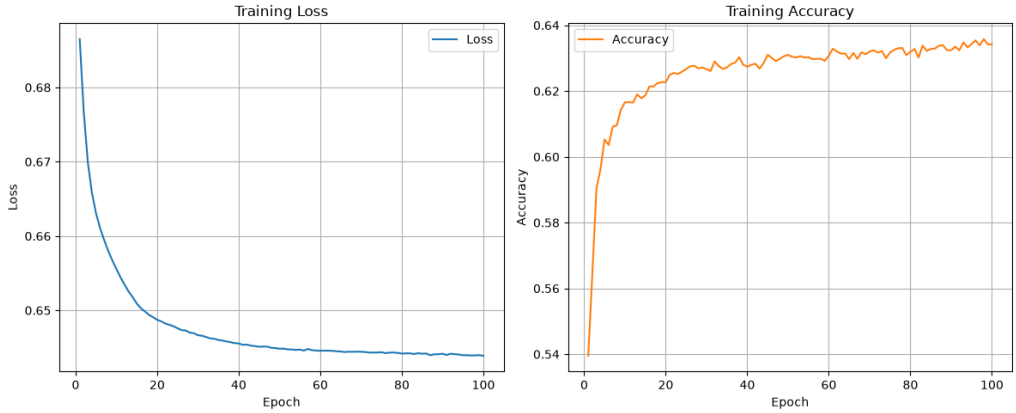


Fig. 1. This figure shows the CIFAR-10 dataset run through the circuit for 100 epochs. The training loss is not shown truly converging within the number of epochs used during this run. The training accuracy appears to converge around 60 epochs at 0.63.

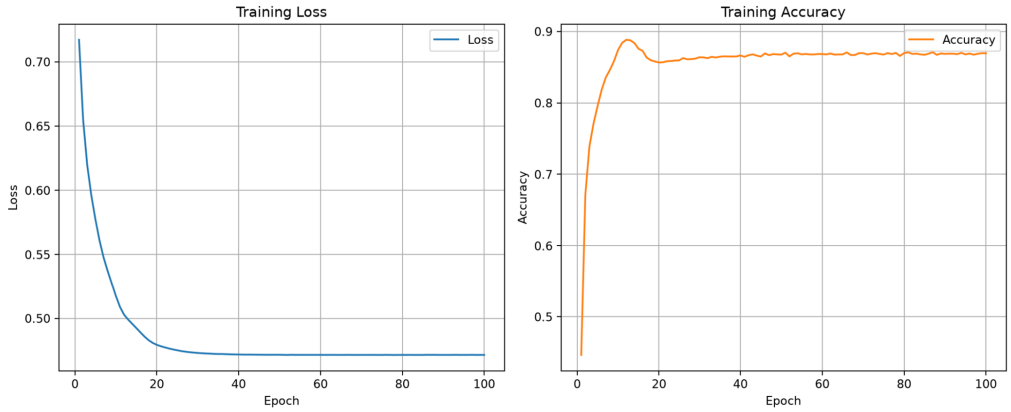


Fig. 2. This figure shows the MNIST dataset run through the circuit for 100 epochs. The training loss appears to converge around 40 epochs at 0.42. The training accuracy appears to converge around 40 epochs as well at 0.97.

B. QNN Model Training with Measurement Noise

In this section, we consider a more realistic setting, where we have considered measurement noise. In practice, near-term quantum hardware is prone to noise and thus QNNs are affected by measurement noise because quantum measurements are inherently probabilistic. This introduces uncertainty in the estimated expectation values, model predictions, and gradients during training. Such fluctuations can reduce prediction accuracy, affect training stability, and slow down model convergence, particularly when a limited number of measurement shots is used. **Our method for reducing statistical measurement uncertainty is to increase the number of measurement shots.** Each shot represents a single execution and measurement of a quantum circuit. By performing multiple shots and averaging the measurement outcomes, QNNs can obtain more accurate estimates of expectation values. Increasing the number of shots reduces statistical fluctuations, leading to more reliable predictions and improved training stability. However, this approach also increases computational cost and quantum hardware execution time.

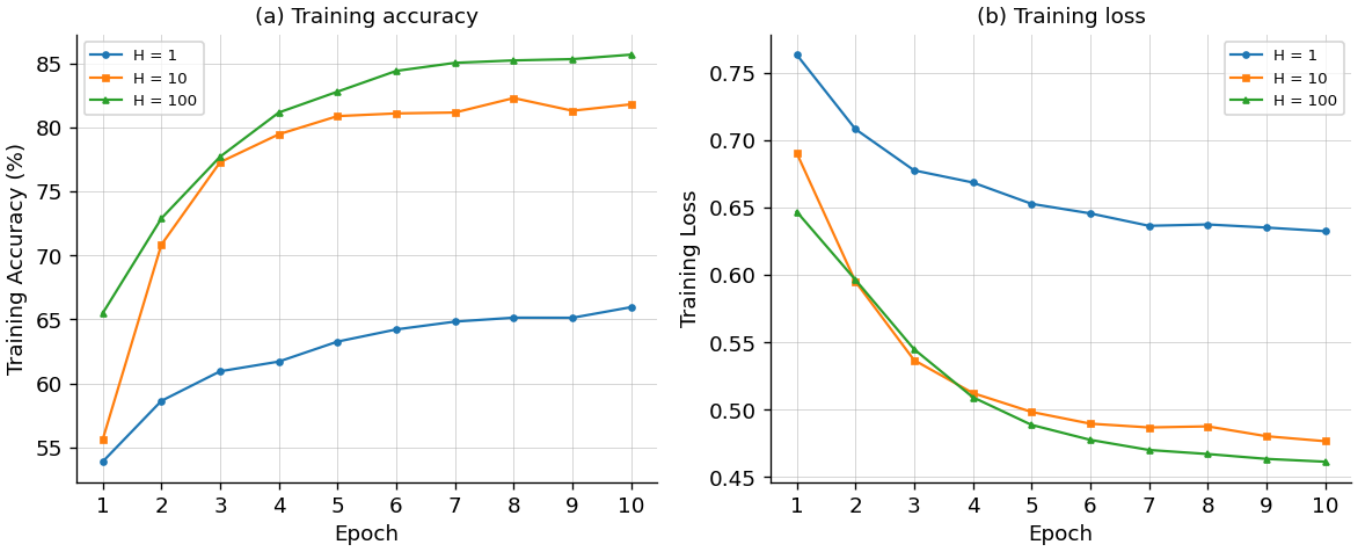


Fig. 3. Training performance of QNN under measurement noise.

As shown in Fig. 3, on MNIST dataset (IID), showcasing improved accuracy with increasing number of quantum measurement shots under full device participation scenario. Fig. 3 shows the training accuracy for QNN considering full device participation with varying levels of quantum measurement shots (H = 1, H = 10, H = 100). From the plot, it is evident that as the number of quantum measurement shots increases there is a significant enhancement in training accuracy. This improvement is attributed to the reduced quantum shot noise effect achieved through the aggregation of more measurement outcomes, which enhances the reliability and stability of the quantum computations related to model training. Starting from a single measurement shot (H = 1) to a higher count (H = 10), and subsequently increasing further (H = 100),consistently improves the QNN's accuracy. This result indicates that increasing the number of quantum measurement shots effectively reduces the instability and errors introduced by quantum shot noise, thus substantially enhancing the overall training performance in QNN systems. The loss performance of QNN in Fig. 3 is in harmony with the accuracy performance of it in Fig. 3. Both figures demonstrate that increasing the number of quantum measurement shots (H = 1, H = 10, H = 100) results in significant enhancements in performance. Increasing the number of measurement shots from H = 1 to H = 10, and subsequently to H = 100, effectively minimizes the errors and variability inherent in quantum measurement,thereby leading to improved loss metrics. This trend shades light on the crucial role of scaling up the number of quantum measurement shots to achieve optimal training outcomes in QNN setups

III. Conclusion

In conclusion, this demonstrated the feasibility of a hybrid quantum-classical neural network for image classification, achieving approximately 97 percent training accuracy on MNIST and 63 percent on CIFAR-10. Increasing measurement shots improved training stability and accuracy by reducing statistical uncertainty, at the cost of additional computation.

Our GitHub: https://github.com/paulcaplin/Quantum-Chargers-2

IV. References

https://github.com/dinhgit/Quantum-Neural-Networks